Reviewer Pack — Minimal Order of Formal Groups and Circuit Monotone Width In…

Entry e774441291f5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 03:52:40 UTC

Minimal Order of Formal Groups and Circuit Monotone Width Inequality

Entry ID: `e774441291f5` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-06 03:52:40 UTC

1. Conjecture

Field A (mathematical branch): Formal Group Theory **Field B** (complexity object): Boolean Circuit Complexity

Statement:

> For every Boolean circuit C with n inputs, the minimal order of a formal group G associated with the gate structure of C satisfies: Order(G) ≤ k * n^(1/2), where k is a constant.

Rationale (proposer's reasoning):

> Formal groups have been used in algebraic combinatorics to encode structural properties. Their minimal order might reflect the underlying complexity of Boolean circuits. A direct connection between this order and circuit monotone width could provide new insights into circuit complexity lower bounds.

Taxonomy category: `FormalGroup_BooleanCircuit` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `55e2e2bd94c941e9`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For the conjecture regarding the minimal order of formal groups and circuit monotone width inequality, a result is considered supported if for all generated circuits C with n inputs (n ≤ 40), the correlation between the computed minimal order Order(G) and the measured monotone width w_mon(C) yields a linear relationship with a slope k such that Order(G) ≤ k * n^(1/2) AND support_fraction >= 0.8, where support_fraction is the proportion of seeds that satisfy this condition.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `formal group theory AND Boolean circuit monotone width inequality` - `Boolean circuit complexity AND minimal order formal groups` - `circuit monotone width inequality IN formal group theory AND Boolean circuits`

Top relevant hits considered: - [s2:8364ff5de07adbecb8c60ddcbb27a62adec3103b] Discrete Mathematics with Combinatorics - [s2:91a121a8f7abb7033e78eef01a5fe17b072ef6be] The Simplest Query Language That Could Possibly Work

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(n):
        if n == 1:
            return ['0']
        elif n == 2:
            return ['0', '1', 'OR']
        else:
            left = generate_circuit(n // 2)
            right = generate_circuit(n - n // 2)
            return left + right + ['AND']

    def evaluate_circuit(circuit, inputs):
        stack = []
        for gate in circuit:
            if gate == '0':
                stack.append('0')
            elif gate == '1':
                stack.append('1')
            elif gate == 'OR':
                b = stack.pop()
                a = stack.pop()
                stack.append('1' if a == '1' or b == '1' else '0')
            elif gate == 'AND':
                b = stack.pop()
```

```
            a = stack.pop()
            stack.append('1' if a == '1' and b == '1' else '0')
    return stack[0]

def monotone_width(circuit):
    n = len(circuit)
    width = [0] * (n + 1)
    for i in range(n - 1, -1, -1):
        if circuit[i] in ['OR', 'AND']:
            width[i] = max(width[i + 1], width[i + 2])
        else:
            width[i] = 1
    return width[0]

def formal_group_order(circuit):
    n = len(circuit)
    order = [0] * (n + 1)
    for i in range(n - 1, -1, -1):
        if circuit[i] in ['OR', 'AND']:
            order[i] = max(order[i + 1], order[i + 2])
        else:
            order[i] = 1
    return sum(order) / n

def run_experiment(n):
    circuit = generate_circuit(n)
    inputs = [random.choice(['0', '1']) for _ in range(n)]
    result = evaluate_circuit(circuit, inputs)
    width = monotone_width(circuit)
    order = formal_group_order(circuit)
    return width, order

n_values = [5, 10, 15, 20, 30, 40]
total_instances = 0
total_order = 0
max_n = 0
conjecture_holds = True
counterexample = ""

for n in n_values:
    for _ in range(5):  # Ensure at least 30 instances per seed
        width, order = run_experiment(n)
        total_instances += 1
        total_order += order
        max_n = max(max_n, n)
        if order > math.sqrt(n):
            conjecture_holds = False
            counterexample = f"Order(G)={order} > sqrt({n}) for n={n}"

mean_order = total_order / total_instances
support_fraction = 1.0 if conjecture_holds else 0.0

return {
    "metric_name": "Formal Group Order",
    "metric_value": mean_order,
    "instances_tested": total_instances,
```

```
        "n_max": max_n,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_order = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_order} std=0.0 support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_order} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{r['counterexample']}\" first_failing_seed={fir
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1ca3fa22.py", line 111, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1ca3fa22.py", line 85, in run_trial
    width, order = run_experiment(n)
                   ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1ca3fa22.py", line 72, in run_experimen
    width = monotone_width(circuit)
            ^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1ca3fa22.py", line 53, in monotone_widt
    width[i] = max(width[i + 1], width[i + 2])
                                 ~~~~~^^^^^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed before producing data, which means it did not complete its execution to provide a result for the conjecture. | next: Re-run the experiment with increased robustness checks and ensure that the code handles all possible edge cases to prevent crashes.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15040
2	preregistration	ollama_remote	glm4:latest	0	0	10091
3	novelty	ollama_remote	glm4:latest	0	0	13773
4	novelty	ollama_remote	glm4:latest	0	0	9613
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16515
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8447
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10491
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13135
9	judge	ollama_remote	glm4:latest	0	0	11994

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 109099 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/e774441291f5.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e774441291f5.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/e774441291f5.tar.gz` (if generated)